CareTrace: A Neurosymbolic Approach to Pediatric After-Hours Triage

Author Name 1
UC Berkeley School of Information
author1@berkeley.edu

Author Name 2
UC Berkeley School of Information
author2@berkeley.edu

Author Name 3
UC Berkeley School of Information
author3@berkeley.edu

DATASCI 290 — Neurosymbolic AI | UC Berkeley School of Information | Spring 2026

ABSTRACT

After-hours pediatric triage is a high-stakes setting in which caregivers seek guidance for acutely ill children without immediate access to clinicians. Large language models (LLMs) are increasingly capable of producing plausible clinical language, but pure generative approaches lack verifiable reasoning provenance, cannot guarantee adherence to established clinical practice guidelines (CPGs), and risk fabricating medication dosing or epidemic context. **CareTrace** is a neurosymbolic prototype that grounds pediatric triage in the Seattle Children's Fever CPG by combining (1) an LLM or rule-based natural language interpretation layer, (2) a SNOMED CT–backed knowledge graph in Neo4j AuraDB populated from the CPG's red-flag criteria, (3) deterministic PyDatalog triage rules with full audit trace, and (4) a provenance-annotated explanation agent. A LangGraph orchestration pipeline sequences these components into a multi-turn conversational system. On three manually crafted scenarios spanning home management, emergency escalation, and urgent same-day care, CareTrace produces correct dispositions with full symbolic justification, while a vanilla LLM baseline cannot guarantee structured CPG adherence. The architecture demonstrates how neurosymbolic integration can improve safety, transparency, and auditability in conversational clinical decision support.

1. INTRODUCTION

1.1 Motivation

Pediatric fever is one of the most common reasons families contact medical advice lines after hours. Guidelines such as the Seattle Children's Fever CPG define clear escalation criteria — seizure, altered consciousness, breathing difficulty, extreme fever, infant under three months — that must be applied consistently. Parents describing symptoms in natural language often omit details critical for safe triage, and any system that fails to elicit those details risks either over-escalating (unnecessary ER visits) or under-escalating (missed emergencies).

Pure LLMs present two risks: (a) CPG thresholds cannot be guaranteed because reasoning is implicit in weights, and (b) they can hallucinate — generating plausible-sounding but incorrect medication dosing, epidemiological context, or age-specific contraindications. CareTrace demonstrates neurosymbolic integration at prototype scale, bounded to febrile illness, GI symptoms, and dehydration risk.

1.2 Contributions

1. **Neurosymbolic triage pipeline:** LangGraph state machine sequencing LLM interpretation, SNOMED-grounded KG retrieval, PyDatalog symbolic rules, and templated explanation with provenance strings.
2. **CPG-derived mini knowledge graph:** SNOMED CT concepts seeded from the Seattle Children's Fever CPG stored in Neo4j AuraDB with `IS_A` hierarchy and attribute edges.
3. **Audit-traceable reasoning:** every disposition includes a named `rule_ids` set explaining exactly which symbolic predicate fired.
4. **Medication safety hooks:** CPG citations for acetaminophen and ibuprofen age gates with provenance strings.
5. **Logic-Augmented Generation (LAG):** explanation mode that constrains the LLM to express symbolic reasoning rather than generate its own.
6. **Comparative baseline:** vanilla LLM baseline without symbolic gates for provenance and consistency evaluation.

2. BACKGROUND

2.1 Neurosymbolic AI

Neurosymbolic AI combines pattern-recognition strengths of neural networks with deductive reasoning guarantees of symbolic systems. Neural components handle variability and ambiguity of natural language; symbolic components enforce hard constraints derived from domain knowledge. LLM-plus-symbolic frameworks have emerged for structured document question answering and clinical guideline compliance.

2.2 Clinical Practice Guidelines and Pediatric Triage

The Seattle Children's Fever CPG specifies fever thresholds, escalation triggers, and medication guidance aligned with the American Academy of Pediatrics. Encoding CPG logic as machine-executable predicates is established in CDSS. CareTrace applies this in a conversational setting where intake fields must be elicited dynamically from unstructured caregiver language.

2.3 Knowledge Graphs for Medical Concept Grounding

SNOMED CT has over 350,000 active concepts organized through `IS_A` and attribute relationships. Snowstorm exposes FHIR-compliant REST APIs for SNOMED lookup. Neo4j is widely used in biomedical KG research. Combining Snowstorm concept retrieval with Neo4j persistence enables compact, query-friendly SNOMED subgraphs grounding natural language to standard clinical identifiers.

3. SYSTEM ARCHITECTURE

3.1 Overview

```
caretrace/
  main.py              ← CLI entrypoint
  config.py            ← Settings / .env loading
  state.py             ← CareTraceState, CaseFields,
                         TriageDecision
  orchestration/graph.py← LangGraph pipeline
  agents/interpretation ← Text → CaseFields
  agents/explanation    ← Decision → reply (LAG)
  agents/medication     ← CPG medication guidance
  logic/triage_rules.py ← PyDatalog rules
  graph/snomed_retrieval← Neo4j SNOMED lookup
  graph/fever_cpg_mentions← CaseFields → KG terms
  graph/neo4j_client    ← Driver + run_cypher
  evaluation/harness    ← Scenario replay
KG_implementation/
  Scaffold_KG_FeverCPG_Text_to_AuraDB.ipynb
  Pediatric_Fever_KG_Snowstorm_to_AuraDB.ipynb
```

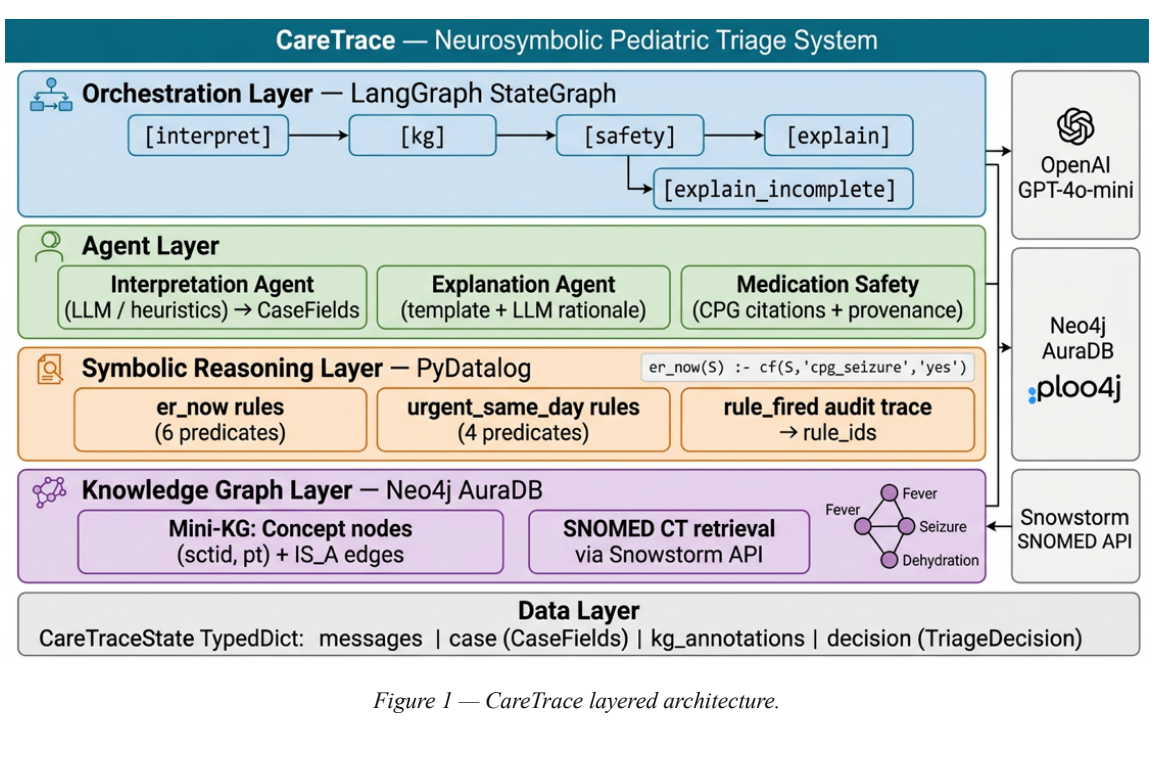
CareTrace — Neurosymbolic Pediatric Triage System
Orchestration Layer — LangGraph StateGraph
[interpret]
[kg]
[safety]
[explain]
[explain_incomplete]
OpenAI GPT-4o-mini
Agent Layer
Interpretation Agent (LLM / heuristics) → CaseFields
Explanation Agent (template + LLM rationale)
Medication Safety (CPG citations + provenance)
Neo4j AuraDB
neo4j
Symbolic Reasoning Layer — PyDatalog
er_now(S) :- cf(S,'cpg_seizure','yes')
er_now rules (6 predicates)
urgent_same_day rules (4 predicates)
rule_fired audit trace → rule_ids
Knowledge Graph Layer — Neo4j AuraDB
Mini-KG: Concept nodes (sctid, pt) + IS_A edges
SNOMED CT retrieval via Snowstorm API
Fever
Seizure
Dehydration
Snowstorm SNOMED API
Data Layer
CareTraceState TypedDict: messages | case (CaseFields) | kg_annotations | decision (TriageDecision)

Figure 1 — CareTrace layered architecture.

3.2 State Model

All per-session data flows through a single `CareTraceState` TypedDict shared across every LangGraph node.

Table 1 — CareTraceState pipeline fields

Field	Type	Purpose
`messages`	`list[dict]`	Full chat history
`raw_user_text`	`str`	Latest caregiver utterance
`case`	`CaseFields`	Canonical clinical fields, accumulated across turns
`kg_annotations`	`list[dict]`	SNOMED concept + ancestor rows from Neo4j
`decision`	`TriageDecision`	Disposition, rule_ids, med_flags, missing_required
`assistant_reply`	`str`	Final reply rendered to caregiver
`turn`	`int`	Turn counter

`CaseFields` — 14-field schema, all independently extractable:

Table 2 — CaseFields schema

Field	Allowed Values / Notes
`temp_f`	Numeric °F or `None`
`temp_unknown`	`bool` — caregiver confirmed unknown
`age_years` / `age_months`	float; months used for infant gates
`weight_kg`	float — required for dose calculation
`alertness`	`normal|sleepy_ok|altered|unknown`
`breathing`	`normal|tachypnea_concern|distress|unknown`
`fluid_intake`	`good|some|poor|none|unknown`
`urine_last_8h`	`yes|no|unknown`
`vomiting`	`none|once|repeated|unknown`
`seizure`	`yes|no|unknown`
`fever_duration_hours`	float; ≥72 h → 3-day CPG rule
`current_meds`	list[str] e.g. `["amoxicillin"]`
`local_outbreak_context`	str|None — never overrides rules

`TriageDecision` — structured output of the safety node:

```
@dataclass
class TriageDecision:
  disposition: str   # ER_NOW | URGENT_SAME_DAY
                     # | HOME_MANAGEMENT | OUT_OF_SCOPE
  rule_ids: list[str]          # e.g. ["R_CPG_SEIZURE"]
  missing_required: list[str]  # [] when intake complete
  med_flags: list[str]         # age/dehydration flags
  out_of_scope_reason: str | None
```

Multi-turn accumulation — `_merge_non_empty`: Each turn extracts a delta dict from the current utterance. Fields with `None`, empty string, empty list, or `"unknown"` (for categorical fields) are excluded — a caregiver omitting temperature in turn 2 does not overwrite the value reported in turn 1:

```
def _merge_non_empty(base, delta) -> CaseFields:
    out = dict(base)
    for k, v in delta.items():
        if v is None: continue
        if isinstance(v, str) and v == "": continue
        if k in CATEGORICAL and v == "unknown": continue
        if isinstance(v, list) and len(v) == 0: continue
        out[k] = v   # overwrite only with real values
    return out
```

3.3 LangGraph Orchestration Pipeline

The graph is built once at import time via `build_app()` and reused across turns:

```
def build_app():
    g = StateGraph(CareTraceState)
    g.add_node("interpret",          node_interpret)
    g.add_node("kg",                 node_kg)
    g.add_node("safety",             node_safety)
    g.add_node("explain",            node_explain)
    g.add_node("explain_incomplete", node_explain_incomplete)
    g.set_entry_point("interpret")
    g.add_edge("interpret", "kg")
    g.add_edge("kg",        "safety")
    g.add_conditional_edges(
        "safety", route_after_safety,
        {"explain": "explain",
         "explain_incomplete": "explain_incomplete"},
    )
    g.add_edge("explain",            END)
    g.add_edge("explain_incomplete", END)
    return g.compile()
```

The routing function inspects `decision.missing_required`:

```
def route_after_safety(state):
    miss = (state.get("decision") or {}) \
             .get("missing_required") or []
    return "explain_incomplete" if miss \
           else "explain"
```

Node `interpret` — dispatches to LLM structured extraction (`ChatOpenAI.with_structured_output(ExtractedCase)`) or heuristic regex extraction, then merges into prior `CaseFields` via `_merge_non_empty`.

Node `kg` — skips when `CARETRACE_SKIP_NEO4J=1`. Otherwise calls `kg_mentions_from_case_and_text` to derive search terms, then runs `annotate_case_mentions` (concept lookup + ancestor traversal).

Node `safety` — pure Python + PyDatalog, no I/O. Calls `required_missing(case)` then `evaluate_triage(case, missing)`.

Node `explain` / `explain_incomplete` — both call `explain(settings, case, decision, kg)`. Dispatches on disposition: incomplete intake → question list; complete → full disposition reply with rule trace, medication guidance, safety netting.

3.4 Interpretation Agent — Dual-Mode Extraction

The Pydantic schema `ExtractedCase` defines the contract for both paths:

```
class ExtractedCase(BaseModel):
    temp_f:       float | None = None
    temp_unknown: bool = False
    age_years:    float | None = None
    age_months:   float | None = None
    weight_kg:    float | None = None
    vomiting:    str = "unknown" # none|once|repeated
    alertness:   str = "unknown" # normal|sleepy_ok|altered
    breathing:   str = "unknown" # normal|tachypnea|distress
    fluid_intake: str = "unknown"# good|some|poor|none
    urine_last_8h: str = "unknown"
    current_meds: list[str] = []
    seizure:      str = "unknown"
    fever_duration_hours: float | None = None
```

LLM path — `ChatOpenAI.with_structured_output(ExtractedCase)` instructs the model via function-calling. System prompt: *"Use unknown when not stated. Never invent vitals. Map qualitative phrases conservatively."*

Heuristic path — representative patterns:

```
# Temperature: "103.5 F", "101°F"
re.search(r"(\d{2,3}(?:\.\d+)?)\s*(?:°?\s*f|f\b)", t)

# Alertness
if any(x in t for x in ("barely responding",
       "won't wake","lethargic")):
    out["alertness"] = "altered"
elif any(x in t for x in ("tired but","answers")):
    out["alertness"] = "sleepy_ok"

# Vomiting — unquantified → conservative "once"
if re.search(r"\b([2-9]|\d{2,})\s*times\b", t):
    out["vomiting"] = "repeated"
elif any(x in t for x in ("once","one time")):
    out["vomiting"] = "once"
elif "vomit" in t or "threw up" in t:
    out["vomiting"] = "once"   # conservative

# Fever duration
m = re.search(r"fever[^\n]{0,40}(\d{1,2})\s*days?",t)
if m: out["fever_duration_hours"]=float(m.group(1))*24
```

3.5 Symbolic Triage Rules — PyDatalog

Rules are defined once at module import. Session-scoped facts (`cf(session_id, name, value)`) prevent cross-session contamination.

ER hard gates:

```
er_now(S) <= cf(S, "alertness",          "altered")
er_now(S) <= cf(S, "breathing",          "distress")
er_now(S) <= cf(S, "dehydration_severe", "yes")
er_now(S) <= cf(S, "fluid_intake","none") \
           & cf(S, "urine_last_8h","no")
er_now(S) <= cf(S, "cpg_seizure",        "yes")
er_now(S) <= cf(S,"cpg_infant_under_3mo_fever","yes")
```

Urgent same-day patterns:

```
urgent_same_day(S) <= cf(S,"temp_f","very_high") \
                    & ~er_now(S)
urgent_same_day(S) <= cf(S,"vomiting","repeated") \
                    & cf(S,"fluid_intake","poor") \
                    & ~er_now(S)
urgent_same_day(S) <= \
    cf(S,"cpg_fever_duration_3_days","yes") \
    & ~er_now(S)
```

Home management:

```
home_candidate(S) <= cf(S,"alertness","normal") \
    & cf(S,"breathing","normal") \
    & ~cf(S,"fluid_intake","none") \
    & ~er_now(S) & ~urgent_same_day(S)
```

Temperature is bucketed in `_case_to_facts`: `very_high` (≥104°F), `high` (≥103°F), `non_extreme`. CPG predicates (`cpg_infant_under_3mo_fever`, `cpg_fever_duration_3_days`) are derived from numeric age and temperature thresholds before assertion. Every rule has a parallel `rule_fired(S, "R_ID")` predicate for audit tracing.

3.6 Knowledge Graph Construction

Bounded BFS expansion (`MAX_TOTAL_CONCEPTS=45`) using Snowstorm REST API:

1. Resolve CPG bullet phrases to SCTIDs via `/MAIN/concepts?term=...`.
2. BFS over `IS_A` parents via `/MAIN/concepts/{id}/ancestors`.
3. Add up to `MAX_ATTR_NEIGHBORS=3` attribute relationship targets per concept.
4. Second pass: ancestor closure on any newly added attribute targets.

Idempotent Neo4j load via `MERGE`:

```
MERGE (c:Concept {sctid: $sctid})
  ON CREATE SET c.pt=$pt, c.kg_source=$src
  ON MATCH  SET c.pt=$pt
// IS_A edge
MATCH (child:Concept {sctid:$child_id})
MATCH (par:Concept   {sctid:$par_id})
MERGE (child)-[:IS_A]->(par)
// Attribute edge
MERGE (src)-[:REL {typeId:$typeId,
                   typeTerm:$typeTerm}]->(tgt)
```

3.7 Schema-Adaptive SNOMED Retrieval

Schema is probed once per driver instance and cached:

```
CALL db.labels()        YIELD label       → frozenset
CALL db.propertyKeys() YIELD propertyKey → frozenset
CALL db.relationshipTypes() YIELD ...    → frozenset
```

Three-schema dispatch for concept lookup:

```
def find_concepts_by_term(driver, term):
    keys   = _cached_property_keys(driver)
    labels = _cached_labels(driver)

    # 1. CPG mini-KG: pt + sctid/conceptId
    if "pt" in keys and ("sctid" in keys
                         or "conceptId" in keys):
        hits = _find_concepts_pt_slice(...)
        if hits or "Description" not in labels:
            return hits

    # 2. Snowstorm RF2: Description label
    if "Description" in labels:
        hits = _find_concepts_snowstorm(...)
        if hits: return hits

    # 3. Legacy: term field
    if "term" in keys:
        return _find_concepts_legacy(...)
    return []
```

Mini-KG concept search Cypher:

```
MATCH (c:Concept)
WHERE c.pt IS NOT NULL
  AND (c.sctid IS NOT NULL
       OR c.conceptId IS NOT NULL)
  AND toLower(toString(c.pt))
      CONTAINS toLower($term)
RETURN coalesce(toString(c.sctid),
                toString(c.conceptId)) AS id,
       toString(c.pt) AS term
LIMIT $limit
```

Ancestor traversal with dynamic `RETURN` clause:

```
MATCH (c:Concept)-[:IS_A*1..8]->(a:Concept)
WHERE {dynamic_id_match_clause}
RETURN DISTINCT
  coalesce(toString(a.sctid),
           toString(a.conceptId)) AS id,
  toString(a.pt) AS term
LIMIT 40
```

3.8 Medication Safety Module

```
@dataclass(frozen=True)
class MedGuidance:
    text: str
    provenance: str   # always includes CPG URL
    must_escalate: bool
```

Decision logic:

- **Acetaminophen:** `must_escalate=True` if age <3 months; refuse to compute dose if `weight_kg` is `None`; otherwise illustrative 15 mg/kg with verification disclaimer.
- **Ibuprofen:** `must_escalate=True` if age <6 months; advise against if dehydration risk present.

PyDatalog `med_flag` predicates mirror the same age gates in the rule trace so medication constraints appear in both `TriageDecision.med_flags` and the natural language explanation.

3.9 AGENT DESIGN

Table 3 — Agent overview

Agent	Input	Output	Modes
Interpretation	Text + prior CaseFields	Updated CaseFields	LLM / Heuristic
Medication Safety	CaseFields	MedGuidance objects	Deterministic only
Explanation	CaseFields + TriageDecision + kg_annotations	Reply string	Template / LLM / LAG

3.9.1 Interpretation Agent

Key design constraint: *conservatism*. If breathing is not mentioned, it stays `"unknown"` rather than defaulting to `"normal"`. This forces the required intake policy to prompt the caregiver, preventing silent assumptions from bypassing safety gates. The LLM path enforces the same conservatism via system prompt and the Pydantic schema's vocabulary constraints enforced through function-calling.

3.9.2 Medication Safety Agent

Entirely deterministic — no LLM, no heuristics. Design principle: *provenance completeness*. Every piece of medication guidance includes a citation string pointing to the exact CPG source URL. The agent explicitly **refuses to compute** doses when `weight_kg` is absent, directly addressing the LLM baseline failure mode of speculative dose computation.

3.9.3 Explanation Agent — Three Execution Modes

Table 4 — Explanation agent execution modes

Mode	Trigger	Mechanism
Template	`CARETRACE_MOCK_LLM=1`	String templates; rule trace and CPG summary inserted; fully deterministic
LLM	API key set, no LAG flag	LLM receives `case`+`decision` dict; free-form reply with constrained system prompt
LAG	`CARETRACE_USE_LAG=1`	LLM receives complete symbolic reasoning chain; constrained to *express*, not reason

Logic-Augmented Generation (LAG) is the most novel aspect. Instead of asking the LLM to reason about the case, it receives the complete symbolic reasoning chain and is asked only to express it in natural language. The context block includes patient state, KG evidence, and every rule evaluated — both fired and not fired:

```
=== SYMBOLIC TRIAGE CONTEXT ===
PATIENT STATE:
- Age: 6 years
- Temperature: 103.5°F [high (≥103°F)]
- Alertness: ALTERED — not responding normally
- Fluid intake: NONE (not drinking)
- Urination last 8h: NO (dry for 8+ hours)

KG EVIDENCE:
  - Fever [SNOMED 386661006]
  - Dehydration [SNOMED 34095006]

TRIAGE RULES EVALUATED:
✓ FIRED: R_ER_ALERTNESS
  Condition: alertness == altered
  CPG: child not alert when awake → ER immediately
✓ FIRED: R_ER_DEHYDRATION_SEVERE
  Condition: poor/no fluid AND no urine
✗ NOT FIRED: R_CPG_SEIZURE
✗ NOT FIRED: R_URGENT_VERY_HIGH_FEVER

FINAL SYMBOLIC DECISION: ER_NOW

=== YOUR TASK ===
Express the symbolic reasoning as a clear,
caregiver-facing explanation. Constraints:
1. State disposition first — do NOT change it.
2. Explain WHY using fired rule labels.
3. Acknowledge rules NOT fired (what was ruled out).
4. Do NOT add content not in the context above.
5. Do NOT invent diagnoses or dosages.
```

Providing both fired and not-fired rules lets the LLM generate explanations that acknowledge explicitly evaluated and cleared red flags, improving caregiver trust. All three modes fall back gracefully to template on any exception.

Table 5 — Rule metadata in LAG context block

Rule ID	Human Label	CPG Basis
`R_ER_ALERTNESS`	Altered alertness	Not alert when awake → ER immediately
`R_ER_BREATHING`	Breathing distress	Trouble breathing → ER immediately
`R_ER_DEHYDRATION_SEVERE`	Severe dehydration	No urine 8h + poor intake → ER immediately
`R_CPG_SEIZURE`	Febrile seizure	Febrile convulsion → ER immediately
`R_CPG_INFANT_UNDER_3MO_FEVER`	Infant <3 months with fever	Fever in infant <3 months → ER immediately
`R_URGENT_VERY_HIGH_FEVER`	Very high fever ≥104°F	Fever ≥104°F → same-day urgent
`R_URGENT_REPEATED_VOMIT_POOR_FLUID`	Repeated vomiting + poor intake	Vomits often + poor intake → same-day urgent
`R_URGENT_FEVER_OVER_3_DAYS`	Fever >3 days	Fever >3 days → same-day urgent
`R_HOME_CONSERVATIVE`	Conservative home management	Alert, breathing normally, taking fluids — no red flags

4. KNOWLEDGE GRAPH DESIGN

4.1 Schema

```
(:Concept {sctid, pt, kg_source})
  -[:IS_A]->(:Concept)
  -[:REL {typeId, typeTerm}]->(:Concept)
```

4.2 Seed Concepts from CPG Extraction

Table 6 — CPG-seeded SNOMED CT anchor concepts

Query	SCTID	Preferred Term
fever	386661006	Fever
seizure	91175000	Seizure
dyspnea	267036007	Dyspnea
tachypnea	271823003	Tachypnea
lethargy	214264003	Lethargy
oliguria	83128009	Oliguria
dehydration	34095006	Dehydration
vomiting	422400008	Vomiting

4.3 Bounded Graph Expansion

BFS up to `MAX_TOTAL_CONCEPTS=45`, following `IS_A` parents and up to `MAX_ATTR_NEIGHBORS=3` attribute edges per concept for `MAX_HOPS_ATTR=1` hops. A second ancestor-closure pass adds remaining `IS_A` parents without expanding attribute edges, ensuring semantic coherence without unbounded growth.

4.4 Runtime Retrieval

At each turn the KG node calls `kg_mentions_from_case_and_text(case, raw_text)` to produce search terms (combining structured case signals with substring cues from the utterance), queries Neo4j with case-insensitive `CONTAINS` on `pt`, and traverses `IS_A` edges up to 8 hops for semantic context.

5. EVALUATION

5.1 Scenario Design

Table 7 — Evaluation scenarios

ID	Clinical summary	Expected
S1	6 yo, 101.8°F, tired but responsive, sipping, on amoxicillin, one emesis, urinated this evening	HOME_MANAGEMENT
S2	6 yo, 103.5°F, barely responding, not drinking, no urine since afternoon	ER_NOW
S3	5 yo, 102.5°F, vomited 4×, only sipping, urinated 1 hour ago	URGENT_SAME_DAY

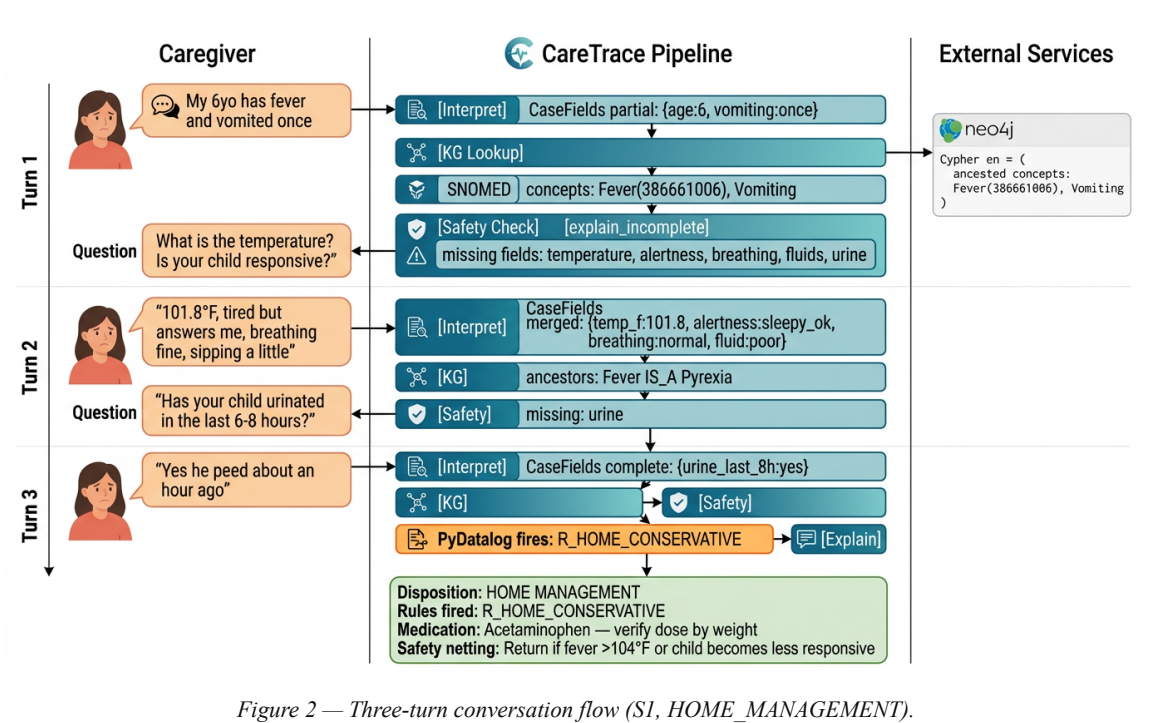
Caregiver
CareTrace Pipeline
External Services
Turn 1
My 6yo has fever and vomited once
[Interpret] CaseFields partial: {age:6, vomiting:once}
[KG Lookup]
SNOMED concepts: Fever(386661006), Vomiting
neo4j
Cypher en = (ancested concepts: Fever(386661006), Vomiting)
Question
What is the temperature? Is your child responsive?
[Safety Check] [explain_incomplete]
missing fields: temperature, alertness, breathing, fluids, urine
Turn 2
"101.8°F, tired but answers me, breathing fine, sipping a little"
[Interpret] CaseFields merged: {temp_f:101.8, alertness:sleepy_ok, breathing:normal, fluid:poor}
[KG] ancestors: Fever IS_A Pyrexia
Question
"Has your child urinated in the last 6-8 hours?"
[Safety] missing: urine
Turn 3
"Yes he peed about an hour ago"
[Interpret] CaseFields complete: {urine_last_8h:yes}
[KG]
[Safety]
PyDatalog fires: R_HOME_CONSERVATIVE
[Explain]
Disposition: HOME MANAGEMENT
Rules fired: R_HOME_CONSERVATIVE
Medication: Acetaminophen — verify dose by weight
Safety netting: Return if fever >104°F or child becomes less responsive

Figure 2 — Three-turn conversation flow (S1, HOME_MANAGEMENT).

5.2 Extracted CaseFields Per Scenario

Table 8 — Final accumulated CaseFields per scenario

Field	S1 (HOME)	S2 (ER_NOW)	S3 (URGENT)
`temp_f`	101.8	103.5	102.5
`age_years`	6	6	5
`alertness`	sleepy_ok	altered	normal
`breathing`	normal	normal	normal
`fluid_intake`	some	none	poor
`urine_last_8h`	yes	no	yes
`vomiting`	once	once	repeated
`current_meds`	[amoxicillin]	[]	[]

5.3 Symbolic Rule Trace

Table 9 — Facts asserted and rules fired per scenario

Scenario	Key facts asserted	Rule IDs fired
S1	alertness=sleepy_ok, breathing=normal, fluid=some	`R_HOME_CONSERVATIVE`
S2	alertness=altered, fluid=none, urine=no, dehydration_severe=yes	`R_ER_ALERTNESS`, `R_ER_DEHYDRATION_SEVERE`
S3	vomiting=repeated, fluid=poor	`R_URGENT_REPEATED_VOMIT_POOR_FLUID`

5.4 Sample System Output (S2 — ER_NOW)

```
Disposition: emergency department now.

Why (rule trace): R_ER_DEHYDRATION_SEVERE,
                  R_ER_ALERTNESS.

Key positives/concerns:
• Reduced responsiveness, hydration, and urine
  output as hard safety signals.
• Local outbreak context does not override gates.

What to do now:
• Go to the ER now; do not wait overnight.
• Bring temperature log and medication list.

Uncertainty: this is not a diagnosis; it is an
escalation based on risk thresholds.
```

5.5 Baseline Comparison

Table 10 — CareTrace vs. vanilla LLM baseline

Dimension	CareTrace	Vanilla LLM
CPG adherence	Guaranteed by PyDatalog rules	Best-effort; varies by phrasing
Disposition provenance	Named `rule_ids` always present	No structured rule citation
Dosing discipline	Weight gate prevents speculative doses	May estimate from age alone
Paraphrase stability	Invariant once fields extracted	May shift on rephrasing
Outbreak context	Recorded; never overrides gates	May use as modifying factor
Intake completeness	Explicit `required_missing`	No structured intake policy

5.6 Required Intake Policy

Table 11 — Mandatory intake fields

Field	Blocking condition if absent
Temperature	`temp_f IS NULL and NOT temp_unknown`
Alertness	`alertness == "unknown"`
Breathing	`breathing == "unknown"`
Fluid intake	`fluid_intake == "unknown"`
Urine last 6–8 h	`urine_last_8h == "unknown"`

6. DESIGN DECISIONS AND TRADE-OFFS

6.1 Why PyDatalog

Python-native Datalog with backward-chaining inference and negation as failure (`~er_now(S)`). Produces enumerable rule traces via `rule_fired` predicates without separate audit infrastructure. Session-scoped facts avoid global state contamination across parallel evaluation runs.

6.2 Why SNOMED CT via Snowstorm

SNOMED CT is the only comprehensive clinical terminology with a publicly available training server (`snowstorm-training.snomedtools.org`). FHIR `CodeSystem/$lookup` for display term resolution ensures terminologically authoritative concept preferred terms.

6.3 Why LangGraph

Typed state machine with conditional routing, node isolation, and compile-time graph validation. Compared to an imperative function chain, orchestration flow is explicit, future nodes can be added without rearchitecting, and node inputs/outputs are cleanly separated.

6.4 Scope Boundaries

Not yet encoded: localized pain (neck, ears, throat), rash, immunocompromised status, "3 months–2 years with fever >24h and no other symptoms," and chronic immune conditions. Documented as extension points in `Docs/CPG_Fever_Seattle_Childrens_reference.md`.

7. RELATED WORK

Clinical decision support: MYCIN [1] established symbolic rule-based diagnosis. Modern CDSS use hybrid rule-plus-retrieval. CareTrace differs in caregiver-facing conversational triage with explicit CPG provenance tracing.

LLM safety in healthcare: Singhal et al. [2] and Nori et al. [3] demonstrate GPT-4 on medical licensing exams but also document hallucination risk in dosing and contraindication scenarios — exactly the categories CareTrace's symbolic layer guards.

Medical knowledge graphs: Rotmensch et al. [4] built EHR-derived disease graphs. CareTrace's contribution is using a CPG-text–seeded SNOMED subgraph as a *runtime* retrieval oracle rather than a training resource.

Neurosymbolic integration: Garcez & Lamb [5] survey neural-symbolic computing. The *interpret* → *symbolic* → *explain* pipeline mirrors the NSCL approach: neural perception feeds symbolic reasoning which feeds interpretable output.

8. LIMITATIONS AND FUTURE WORK

Limitations:

- Evaluation limited to three manually authored scenarios; larger-scale clinical validation required.
- KG bounded to ~45 SNOMED concepts; rare symptom combinations may not be grounded.
- Heuristic interpretation fails on phrasing outside its regex vocabulary.
- Course prototype: thresholds and escalation logic require licensed clinician review before deployment.

Future directions:

- Extend `CaseFields` and rules for rash, immunocompromise, localized pain.
- Add `kg_rag` node surfacing `kg_annotations` into LLM-backed explanation prompts.
- Fine-tune structured-extraction model on synthetic triage transcripts.
- Multi-CPG support: RSV, croup, urinary tract infections.

9. CONCLUSION

CareTrace demonstrates that neurosymbolic integration is both feasible and beneficial for conversational pediatric triage. By combining LLM-based natural language understanding with PyDatalog symbolic rules derived from the Seattle Children's Fever CPG, SNOMED CT knowledge graph grounding in Neo4j, and CPG-cited medication guidance, the system produces structured, auditable, and clinician-legible triage dispositions. The architecture's key properties — traceable rule firing, explicit required-intake policy, provenance-annotated medication language, and Logic-Augmented Generation for explanation — directly address the failure modes of pure generative approaches in safety-critical clinical contexts.

REFERENCES

[1] Shortliffe, E.H. (1976). *Computer-Based Medical Consultations: MYCIN*. Elsevier.

[2] Singhal, K. et al. (2023). Large language models encode clinical knowledge. *Nature*, 620, 172–180.

[3] Nori, H. et al. (2023). Capabilities of GPT-4 on Medical Challenge Problems. arXiv:2303.13375.

[4] Rotmensch, M. et al. (2017). Learning a health knowledge graph from electronic medical records. *Scientific Reports*, 7, 5994.

[5] d'Avila Garcez, A., & Lamb, L.C. (2023). Neurosymbolic AI: The 3rd Wave. *AI Communications*, 36, 1–8.

[6] Seattle Children's. *Fever — Safety and Wellness*. https://www.seattlechildrens.org/health-safety/illness/fever

[7] SNOMED International. *SNOMED CT*. https://www.snomed.org

[8] LangChain Inc. *LangGraph: Build Stateful Multi-Actor Applications*. 2024.

Medical disclaimer: *CareTrace is a course prototype only and is not a substitute for licensed clinical decision support or professional medical advice. All dosing guidance, thresholds, and escalation logic must be reviewed and validated by licensed clinicians before any real-world deployment.*